Matteo Bonanomi
March 27, 2026

LibreChat Project

Contents

1 Linux Machine Configuration **3**

2 Docker configuration **4**

3 Microservices config files explained **5**

3.1 Environment Variables . . . 5

3.2 Librechat configuration file . . . 5

3.3 LiteLLM configuration file . . . 6

3.4 Docker Compose configuration file . . . 7

4 Introduction to RAG in LibreChat **10**

4.1 What a standard RAG is . . . 10

4.1.1 How RAG is implemented in LibreChat . . . 10

4.1.2 Why this is different from "Upload as Text" . . . 10

4.1.3 Why "Upload as Text" can be inefficient when RAG is available . . . 11

4.1.4 Simple example . . . 11

4.1.5 Practical rule of thumb . . . 11

4.2 Standard RAG Parameters . . . 11

4.2.1 `RAG_CHUNK_SIZE=1000` . . . 11

4.2.2 `RAG_CHUNK_OVERLAP=120` . . . 12

4.2.3 `RAG_EMBEDDING_BATCH_SIZE=150` . . . 12

4.2.4 `RAG_EMBEDDING_MAX_QUEUE_SIZE=2` . . . 12

4.2.5 `PDF_EXTRACT_IMAGES=false` . . . 12

4.3 Embedding Migration Scripts . . . 13

5 Reasoning Models **13**

5.1 Reasoning as a routing layer in LiteLLM . . . 13

5.2 How the reasoning tiers are structured . . . 13

5.3 Why this pattern works well . . . 14

5.4 Fine-tuning reasoning in LiteLLM . . . 14

5.5 How to expose reasoning cleanly in LibreChat . . . 15

5.6 Recommended mapping between task types and reasoning levels . . . 15

5.7 Parameter tuning and task sensitivity . . . 16

5.8 A note on context-heavy workflows . . . 16

5.9 Best practices . . . 16

6 Web Search in LibreChat **17**

6.1 Role of SearXNG, Serper, and Jina . . . 17

6.2 Why this decomposition matters 17
6.3 What is important to configure correctly 17
6.4 Recommended operational principles 18

7 Agents in LibreChat **19**
7.1 How Agents should be used in practice 19
7.2 Best practices for designing effective Agents 19
7.3 Model tuning for Agents 20
7.4 When parameter tuning is especially important 20
7.5 Recommended deployment strategy 21
7.6 Operational and security considerations 21
7.7 Templates and examples in this code base 21

1 Linux Machine Configuration

First access to server and basic system update. You should have SSH key already generated and configured. Suggested instance type: ubuntu minimal or another Debian-based distro.

```
ssh root@SERVER_IP_HERE
apt update && apt upgrade -y
apt autoremove -y
apt autoclean
```

install minimal firewall

```
sudo apt install ufw -y
sudo ufw allow OpenSSH
sudo ufw enable
```

Configure Swap (2GB to be added to sugggested 4GB RAM for a basic LibreChat usage):

```
sudo fallocate -l 2G /swapfile
sudo chmod 600 /swapfile
sudo mkswap /swapfile
sudo swapon /swapfile
echo '/swapfile none swap sw 0 0' | sudo tee -a /etc/fstab
```

Optimize Linux kernel for limited RAM usage, limiting aggressive swap usage.

```
echo "vm.swappiness=10" | sudo tee -a /etc/sysctl.conf
sudo sysctl -p
```

Add a new user, avoiding root login

```
adduser deploy
usermod -aG sudo deploy
rsync --archive --chown=deploy:deploy ~/.ssh /home/deploy
```

Test the new user SSH login, then remove SSH login permission for root user, editing the file:

```
sudo nano /etc/ssh/sshd_config
```

Replace the following options:

```
PermitRootLogin no
PasswordAuthentication no
```

Restart SSH and it's done!

```
sudo systemctl restart ssh
```

2 Docker configuration

Docker prerequisites

```
sudo apt install ca-certificates curl gnupg -y
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor
    -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
```

Add repository and install docker

```
echo \
  "deb [arch=$(dpkg --print-architecture)
    signed-by=/etc/apt/keyrings/docker.gpg]
    https://download.docker.com/linux/ubuntu \
  $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install docker-ce docker-ce-cli containerd.io docker-buildx-plugin
    docker-compose-plugin -y
```

Activate Docker daemon and allow user to use Docker

```
sudo systemctl enable docker
sudo usermod -aG docker deploy
```

Fine-tune docker usage for 4GB RAM

```
sudo nano /etc/docker/daemon.json
```

```
{
  "log-driver": "json-file",
  "log-opts": {
    "max-size": "10m",
    "max-file": "3"
  }
}
```

```
sudo systemctl restart docker
```

3 Microservices config files explained

3.1 Environment Variables

Edit the ".env.fake" file and rename it ".env". Paste it inside your project folder. In my repo you can find placeholder to be replaced by databases passwords or API keys. Generate robust passwords using commands like "openssl rand -hex 16".

3.2 Librechat configuration file

The `librechat.yaml` file is the main configuration file of a LibreChat instance. It defines which features are enabled, which model backends are exposed, and how uploaded files are handled. For a first-time LibreChat user, it is best read as the high-level control layer of the application.

Its structure is simple and organized by responsibility:

- `version`: declares the configuration format version used by the file.
- `interface`: enables user-facing features such as `fileSearch` and `fileCitations`.
- `endpoints`: defines the available backends, agent capabilities, and custom model providers.
- `fileConfig`: sets upload limits, token limits, and accepted file types.

The `interface` section enables document-aware interactions:

- `fileSearch` allows users to query uploaded files.
- `fileCitations` allows answers to reference relevant source content.

The `endpoints` section describes how LibreChat connects to AI backends:

- `agents` configures agent behavior through variables such as `disableBuilder`, `maxCitations`, `maxCitationsPerFile`, and `minRelevanceScore`.
- the `capabilities` list includes features such as `file_search` and `context`, meaning that agents can work with retrieved documents and additional contextual information.
- `custom` defines external providers; in this case, LibreChat exposes a LiteLLM endpoint through `apiKey`, `baseURL`, and the list in `models`.

The `fileConfig` section defines file handling rules:

- `fileLimit`, `fileSizeLimit`, and `totalSizeLimit` control how many files can be uploaded and how large they can be.
- `supportedMimeTypes` defines which file types are accepted.
- separate rules can be assigned to `default`, `agents`, and `LiteLLM`, so each context can have different limits.

Some global parameters are especially important:

- `serverFileSizeLimit`: maximum size accepted by the server.
- `avatarSizeLimit`: maximum size for avatar uploads.
- `fileTokenLimit`: maximum text budget extracted from files for downstream processing.

Finally, the `text` subsection refines which file formats can be treated as text-bearing inputs, including PDFs, office documents, markup files, and many source code formats. This is important because it determines which uploaded files can actually participate in retrieval and context-based workflows.

In short, `librechat.yaml` defines three core aspects of a LibreChat deployment:

- which interface features are available;
- which models and agent capabilities are exposed;
- which files can be uploaded, parsed, and used by the system.

3.3 LiteLLM configuration file

This LiteLLM configuration defines how logical model names are mapped to actual upstream providers, how fallback routes are handled, and which global limits apply to the proxy. In this setup, LiteLLM acts as an abstraction layer between client applications and the real model endpoints exposed through OpenRouter.

Its structure is centered around three main sections:

- `model_list`: declares the models exposed by the proxy and maps each alias to its backend configuration.
- `litellm_settings`: defines runtime behaviors such as fallback chains.
- `general_settings`: applies global controls such as authentication and spending limits.

The core idea behind `model_list` is that each `model_name` is a stable internal alias. Client applications can call names such as `qwen_8b` or `llama_70b` without needing to know the exact upstream identifier in `litellm_params.model`. This makes the system easier to maintain, because provider-specific model strings can change without forcing downstream applications to change their own configuration.

Each entry in `litellm_params` defines the connection details used by LiteLLM:

- `model`: the actual upstream model identifier.
- `api_key`: the credential used to authenticate requests.
- `extra_headers`: optional metadata sent to the upstream gateway.

A key design pattern in this file is the *free + paid* pairing. Several models appear in two variants, for example `qwen_8b` and `qwen_8b_paid`. The free variant is typically the preferred default entry point, while the paid variant acts as a backup route.

This logic is made explicit in `litellm_settings.fallbacks`:

- if a request to a free model fails, LiteLLM can automatically retry against the corresponding paid model;
- this improves service continuity without forcing the user to manually switch models;
- the strategy optimizes for lower cost first, while still preserving availability when the free endpoint is rate-limited, unavailable, or unstable.

This is an important operational choice: the system is designed to prefer cheaper access paths by default, but it does not fully rely on them. Instead, it treats paid models as resilience layers. In practice, this is a sensible proxy design for hobbyist, experimental, or small-team deployments that want to balance cost and reliability.

Another relevant element is the presence of an embedding model, here exposed as `bge-m3`. Unlike the chat or instruct models, an embedding model is not mainly used to generate natural-language answers. Its role is to convert text into vector representations that can be compared, indexed, and retrieved efficiently.

In practical terms, an embedding model is useful for tasks such as:

- semantic search;
- document retrieval;

- similarity matching;
- retrieval-augmented generation pipelines.

This means the configuration is not limited to plain chat inference: it also supports information retrieval workflows, which are especially useful in AI applications that work with uploaded documents or external knowledge bases.

The `general_settings` section defines global proxy controls:

- `master_key`: the main authentication key required to access the LiteLLM proxy.
- `max_budget`: a global spending cap that helps constrain overall usage costs.

The presence of `max_budget` is particularly important in shared or semi-public deployments. It introduces a simple budget governance mechanism, helping prevent uncontrolled consumption when multiple users, services, or fallback paths are active.

A broader architectural aspect of this configuration is its clear preference for an open-source or open-weights ecosystem. Even though the models are accessed through a unifying gateway, the selected stack strongly favors openly available model families and interoperable routing logic. This reflects a practical philosophy: keeping the serving layer flexible, reducing lock-in, and making it easier to swap models as the open model landscape evolves.

3.4 Docker Compose configuration file

This `docker-compose` file defines the full runtime architecture of the AI stack. Its purpose is to orchestrate the main application services, their dependencies, the internal network, and the persistent storage required by the platform. Rather than describing a single container, it expresses the whole system as a coordinated set of components.

The overall structure is straightforward:

- `name`: assigns a name to the compose project, here `ai-stack`.
- `services`: defines the application containers and their roles.
- `networks`: defines how containers communicate with each other.
- `volumes`: defines persistent storage used by stateful services.

The design follows a modular service-based logic. Each container has a clear responsibility:

- `mongodb`: stores LibreChat application data, such as users, sessions, and metadata.
- `vectordb`: provides the PostgreSQL + pgvector layer used for retrieval and embeddings storage.
- `litellm`: exposes a unified model proxy for inference and embedding requests.
- `rag_api`: handles document ingestion, chunking, embeddings, and retrieval logic.
- `librechat`: provides the main user-facing chat application.
- `cloudflared`: publishes the local service through a Cloudflare tunnel.

A key architectural point is that the stack separates concerns cleanly:

- conversational state is stored in `mongodb`;
- vector search data is stored in `vectordb`;
- model access is centralized through `litellm`;
- retrieval logic is delegated to `rag_api`;

- the frontend and application layer remain in `librechat`.

This separation makes the system easier to reason about, replace, and scale. Each service can evolve independently as long as its interface remains stable.

The `depends_on` directives express startup relationships between containers:

- `rag_api` depends on `vectordb`;
- `librechat` depends on `mongodb`, `litellm`, and `rag_api`;
- `cloudflared` depends on `librechat`.

This does not fully guarantee service readiness, but it documents the intended dependency graph and ensures a coherent startup order.

The networking model is intentionally simple. All services are attached to the internal bridge network `internal`:

- containers can communicate with each other by service name, for example `mongodb`, `vectordb`, `litellm`, or `rag_api`;
- most services are not directly exposed to the host machine;
- only `librechat` publishes a local port through `127.0.0.1:3080:3080`.

This is an important security-oriented choice: the internal components remain reachable only inside the Docker network, while external exposure is minimized.

The stack also relies heavily on environment variables. This keeps secrets and deployment-specific values outside the compose file itself. Variables such as `LITELLM_MASTER_KEY`, `OPENROUTER_API_KEY`, `JWT_SECRET`, `POSTGRES_PASSWORD`, and `CLOUDFLARE_TUNNEL_TOKEN` externalize credentials and operational parameters. This is a common and sensible pattern for portable deployments.

The `litellm` and `rag_api` services are especially important together. Their interaction defines the retrieval pipeline:

- `rag_api` does not call an external provider directly;
- instead, it points to `OPENAI_BASE_URL` set to `http://litellm:4000/v1`;
- authentication is handled through `RAG_OPENAI_API_KEY` and `OPENAI_API_KEY`, both aligned with `LITELLM_MASTER_KEY`;
- the embedding model is selected through `EMBEDDINGS_MODEL`, here `bge-m3`.

This means LiteLLM acts as a unified internal gateway not only for chat models, but also for embedding generation. It simplifies service integration because downstream components can speak a familiar API format while the actual provider logic remains abstracted.

The retrieval subsystem is further shaped by parameters such as `COLLECTION_NAME`, `CHUNK_SIZE`, `CHUNK_OVERLAP`, `EMBEDDING_BATCH_SIZE`, and `EMBEDDING_MAX_QUEUE_SIZE`. These variables define how documents are segmented, processed, and queued for embedding. In practice, they control the trade-off between retrieval granularity, throughput, and resource usage.

Persistent storage is handled through named volumes:

- `mongo_data`: preserves MongoDB data;
- `rag_pg_data`: preserves pgvector/PostgreSQL data;
- `rag_uploads`: preserves uploaded files handled by the retrieval service.

This is a fundamental part of the stack design, because databases and uploaded documents must survive container restarts or image updates.

Another important aspect is configuration injection through mounted files:

- `./litellm/config.yaml:/app/config.yaml:ro` injects the LiteLLM configuration as read-only;
- `./librechat.yaml:/app/librechat.yaml:ro` injects the LibreChat configuration as read-only.

This keeps application behavior declarative and easy to modify without rebuilding images.

Finally, `cloudflared` adds a controlled publication layer. Instead of exposing the service directly on a public interface, the stack can route traffic through a Cloudflare tunnel. This is often preferable for small self-hosted deployments because it reduces direct surface exposure and simplifies remote access.

In summary, this compose file is built around a few core ideas:

- clear separation between chat application, model proxy, retrieval API, and data stores;
- internal-only communication through the `internal` network;
- persistent state through named volumes;
- centralized model and embedding access through `litellm`;
- configurable retrieval behavior through `rag_api`;
- minimal external exposure, optionally extended through `cloudflared`.

4 Introduction to RAG in LibreChat

4.1 What a standard RAG is

Retrieval-Augmented Generation (RAG) is a method used to improve question answering over documents. Instead of sending the entire content of one or more files directly to the language model, the system first searches the document collection for the most relevant passages. Only those relevant passages are then added to the prompt sent to the model.

In practice, a RAG pipeline usually works as follows:

1. Files are uploaded and processed.
2. The content is split into smaller chunks.
3. Each chunk is converted into a vector embedding.
4. The embeddings are stored in a vector database.
5. When the user asks a question, the same embedding process is applied to the query.
6. The system retrieves the most relevant chunks from the vector database.
7. Only those retrieved chunks are inserted into the model context to support the final answer.

This approach is useful because the model does not need to read the full document every time. It only receives the parts that are most likely to answer the user question.

4.1.1 How RAG is implemented in LibreChat

In LibreChat, RAG is provided through the `RAG API` and the `file_search` capability. When a file is uploaded for file search, LibreChat indexes the document, creates embeddings, and stores them for semantic retrieval. At query time, LibreChat does not inject the full file into the conversation. Instead, it performs retrieval and returns only the most relevant chunks to the model.

This is conceptually different from standard file text injection. In LibreChat there are two different modes:

- **Upload as Text / File Context**: the file content is extracted and included directly in the conversation or in the agent instructions.
- **Upload for File Search**: the file is indexed for semantic retrieval and only relevant chunks are sent to the model when needed.

4.1.2 Why this is different from "Upload as Text"

Uploading a file "as text" is simple and sometimes useful for short documents, but it is not the same as RAG. When the file is uploaded as text, the extracted content is inserted more directly into the context seen by the language model. This means the model may receive a very large amount of text even when the user only needs a small part of it.

When RAG is enabled, the workflow is more efficient:

- the full document is stored as searchable chunks rather than repeatedly pushed into the prompt,
- only the relevant parts are retrieved,
- token usage is reduced,
- prompt cost is lower,
- large documents are easier to handle,
- the context window is used more efficiently.

4.1.3 Why "Upload as Text" can be inefficient when RAG is available

If a large document is uploaded as plain text, most or all of the extracted content may end up in the LLM context. This creates two common problems:

1. **High token cost**: the model must process many tokens that are not relevant to the question.
2. **Context window pressure**: large files can consume a significant part of the available context window, leaving less room for the user query, system instructions, tool results, and the model response.

For this reason, "Upload as Text" is usually better for short files or temporary use, while "Upload for File Search" is usually better for knowledge bases, manuals, reports, policies, technical documentation, and collections of many files.

4.1.4 Simple example

Assume a user uploads a 150-page technical manual and asks:

> "How do I reset the device after a firmware failure?"

With **Upload as Text**, a large amount of the manual may be inserted into the model context, even though only a few paragraphs are actually relevant.

With **RAG / File Search**, LibreChat first searches the indexed manual, finds the chunks related to firmware recovery and device reset, and sends only those passages to the model. The answer is usually more efficient, cheaper, and easier to scale to many documents.

4.1.5 Practical rule of thumb

A simple rule is the following:

- Use **Upload as Text** for small files, quick one-off tasks, or cases where the whole text must be visible to the model.
- Use **Upload for File Search** when your RAG pipeline is online and you want scalable, retrieval-based access to documents.

In most production scenarios, file search is the better default for large or multiple documents because it avoids wasting tokens on irrelevant text and makes better use of the model context window.

4.2 Standard RAG Parameters

The most important parameters to be fine-tuned for your standard RAG systems must be placed in your .env file and will be recalled by the RAG API microservice in the Docker Compose YAML.

4.2.1 `RAG_CHUNK_SIZE=1000`

A good default for general-purpose document collections.

Suggested ranges

- `700--900` for FAQs, manuals, API documentation, and highly specific lookups
- `1000--1200` for mixed-content corpora
- `1200--1600` for narrative or long-form documents where preserving context is more important than pinpoint retrieval

For `bge-m3`, a value of `1000` is a sensible middle ground. Although the model supports very long inputs, larger chunks do not automatically improve retrieval quality.

4.2.2 `RAG_CHUNK_OVERLAP=120`

Also a solid default.

Suggested ranges

- `80--120` for most use cases
- `120--180` for legal, policy, or technical documents where context often crosses section boundaries
- Below `60` only when reducing storage size and indexing cost is a priority

A useful rule of thumb is to keep overlap around `10--15%` of the chunk size. With `120` over `1000`, this configuration is well balanced.

4.2.3 `RAG_EMBEDDING_BATCH_SIZE=150`

Usable, but somewhat aggressive as a general-purpose default.

Suggested ranges

- `16--32` on CPU-only deployments
- `32--64` on modest GPUs
- `64--96` on stronger GPUs
- `150` only if memory usage and throughput have already been validated in the target environment

Higher batch sizes can speed up indexing, but they also increase RAM or VRAM usage. For most installations, `64` is a safer baseline.

4.2.4 `RAG_EMBEDDING_MAX_QUEUE_SIZE=2`

A good conservative value, and generally worth keeping unchanged.

Suggested ranges

- `1--2` for stability
- `3--4` only if the embedding backend and hardware can reliably handle burstier workloads

Larger queue sizes may improve throughput, but they also increase memory pressure and can make overload situations harder to control.

4.2.5 `PDF_EXTRACT_IMAGES=false`

Appropriate for standard text-oriented RAG pipelines.

Set this to `true` only when:

- PDFs contain important charts, diagrams, or figures
- slide decks or image-heavy reports are being indexed
- text extraction alone misses critical information

4.3 Embedding Migration Scripts

The `embedding/` folder contains scripts for managing embedding model migrations in LibreChat with RAG:

- `migrate-embeddings.sh`: Full migration script that backups the pgvector database, stops services, drops old embedding tables (created with the previous model), and restarts services with the new embedding model. Useful when switching between embedding models (e.g., OpenAI 1536-dim to BGE-M3 1024-dim).
- `test-embed.sh`: Simple test script to verify embeddings are being generated correctly via the configured provider (currently LiteLLM routing to OpenRouter's BGE-M3 model).

Both scripts assume Docker Compose is used and interact with the `vectordb` (pgvector) and `rag_api` services.

5 Reasoning Models

A practical way to introduce reasoning into a self-hosted chat stack is not to expose raw provider model names directly to end users, but to create a small number of curated model tiers that correspond to different levels of cost, speed, and cognitive depth. In this code base, reasoning is not presented as a single abstract slider. Instead, it is implemented as a routing policy in LiteLLM and then exposed in LibreChat through human-readable presets.

This design has two advantages. First, it decouples the user experience from the underlying provider-specific model catalog. Second, it makes it possible to align each visible option with a specific operational intent, such as "fast and simple," "balanced reasoning," or "maximum depth."

5.1 Reasoning as a routing layer in LiteLLM

In this architecture, LiteLLM acts as the reasoning control plane. The visible model names used in LibreChat do not map one-to-one to a single provider model; instead, each user-facing route is backed by a primary model and one or more fallbacks. This means that reasoning policy is defined at the proxy layer, not only at the user-interface layer.

Conceptually, this is important because "reasoning level" is often not a single native parameter shared by all providers. In practice, different levels of reasoning are created by combining several controls:

- the choice of the underlying model family and size;
- whether reasoning is explicitly enabled or suppressed at request time;
- timeout and streaming limits;
- fallback chains;
- request budgets and traffic caps.

In other words, reasoning is best understood as a *deployment policy*, not merely as a toggle.

5.2 How the reasoning tiers are structured

A user-friendly reasoning taxonomy should reflect task complexity rather than provider branding. In this project, the visible tiers can be understood as follows.

Small – No Reasoning This tier is intended for very simple prompts, boilerplate tasks, and ultra-fast interactions. It is useful when one wants low latency and low cost, and when explicit chain-of-thought-style behavior is not needed. In the current LiteLLM configuration, this tier is also the one where reasoning is deliberately suppressed.

Small – Light This tier is suitable for lightweight multi-step reasoning, short reformulations, simple classification, quick synthesis, and moderately structured drafting. It offers a reasonable compromise between latency and quality.

Medium – Balanced This is the general-purpose tier for everyday usage. It is appropriate for standard analytical tasks, guided writing, ordinary technical support, structured explanation, and moderate retrieval-augmented interactions.

Medium – Deep This tier is intended for harder synthesis tasks, more nuanced comparisons, and tasks that benefit from stronger deliberate reasoning without necessarily jumping to the most expensive model family.

Large – Balanced This tier uses large-capacity models, but in the current configuration it suppresses reasoning explicitly. This is a useful design pattern when one wants the broad competence of a large model while avoiding unnecessary reasoning overhead, excessive verbosity, or slower behavior.

Large – Max This is the most capable visible tier. It is the appropriate choice for long-form synthesis, difficult planning, multi-source reasoning, complex debugging, or tasks that combine broad context with deliberate inference. In the present stack, this is the clearest "maximum reasoning" route.

5.3 Why this pattern works well

This design is effective because it separates *backend complexity* from *frontend simplicity*. A user sees a small set of understandable choices, while the maintainer preserves full control over model routing. This makes the system easier to operate and easier to evolve. For example, a fallback can be replaced, a timeout can be adjusted, or a primary provider can be changed without forcing users to relearn the interface.

It also creates a more meaningful UX than exposing twenty raw provider models with unclear differences. Most users do not want to choose between many model IDs; they want to choose between outcomes such as *fast*, *balanced*, and *maximum quality*.

5.4 Fine-tuning reasoning in LiteLLM

In LiteLLM, reasoning behavior can be shaped through a combination of parameters and routing choices.

Underlying model selection The most important determinant of reasoning quality is still the model itself. Smaller models are cheaper and faster, but they are usually less robust on long, ambiguous, or multi-step tasks. Medium models are often the best default for daily work. Large models are usually the best fit for synthesis-heavy tasks, difficult reasoning, and web-augmented workflows.

Reasoning enable/disable policy A useful pattern is to explicitly suppress reasoning for routes that are meant to be fast and utilitarian, while enabling it for routes intended for deeper analysis. This makes the visible model catalog more interpretable: users do not need to know implementation details, because the semantic difference between tiers is already encoded in the route.

Timeouts and stream timeouts Timeouts are part of reasoning policy because deeper reasoning usually requires more latency budget. If a route is meant for quick replies, aggressive timeouts are acceptable. If a route is intended for more demanding tasks, longer timeouts improve stability. In practice, these settings should be aligned with the expected complexity of the tier.

Fallback chains Fallbacks are not only an availability feature; they are also part of quality control. A reasoning-oriented route should fall back to models of comparable capability whenever possible. If the fallback chain is too heterogeneous, the same visible model name may behave inconsistently over time.

Request caps and budget controls Traffic controls help preserve system stability. Lightweight tiers can support more concurrent or repeated usage, while premium reasoning tiers should be rate-limited more aggressively. This is a sensible operational policy because the most powerful routes are also the most expensive and computationally demanding.

Context and output limits Even when the provider supports very large contexts, it is unwise to assume that all retrieved or generated content should be passed through unchanged. Input and output limits should be treated as budgeting tools. A high-capacity model is more robust, but it is not immune to prompt bloat.

5.5 How to expose reasoning cleanly in LibreChat

LibreChat should not expose the backend routing logic directly. Instead, it should present each route as a curated option with a readable label and a short description. This is exactly the right place to translate backend complexity into a simple mental model for the user.

A good presentation layer typically includes:

- a short and memorable label;
- a brief description of the intended use case;
- consistent naming across size tiers;
- only a small number of visible choices.

This makes reasoning discoverable without overwhelming the interface. A user can understand the difference between *Small · No Reasoning*, *Medium · Deep*, and *Large · Max* much more quickly than they can understand a list of raw provider model identifiers.

5.6 Recommended mapping between task types and reasoning levels

A practical policy is to assign reasoning tiers to classes of tasks rather than to individuals.

Use small no-reasoning routes for simple reformulations, template filling, quick factual transformations, low-value chat interactions, and any task where latency matters more than deep inference.

Use light or balanced reasoning routes for everyday assistant usage, document summaries, short explanations, ordinary coding help, general knowledge questions, and moderate file-grounded work.

Use deeper reasoning routes for planning, multi-step problem solving, complex comparisons, architectural choices, high-value writing, and ambiguous technical analysis.

Use maximum reasoning routes for web-grounded synthesis, broad-context investigations, difficult debugging, long-form generation, and tasks where multiple sources or constraints must be reconciled carefully.

5.7 Parameter tuning and task sensitivity

Different tasks stress different parts of the system, so reasoning should not be tuned in isolation.

For example, web search and agent workflows often create large prompt assemblies that include instructions, tool traces, retrieved evidence, citations, and prior conversation history. In those cases, the quality of the answer depends not only on model size, but also on context budgeting. A route that works perfectly for ordinary chat may become fragile once web search is enabled if retrieved content is too large.

This is why reasoning design should always be coordinated with:

- retrieval limits;
- reranking strictness;
- context-window budgeting;
- max output length;
- the intended role of the route.

A useful rule of thumb is that stronger reasoning should be paired with stricter prompt discipline, not with looser controls.

5.8 A note on context-heavy workflows

One of the most important practical lessons in this project is that reasoning quality and context management are tightly coupled. A large reasoning-enabled model can still fail if too much low-value retrieval content is injected into the prompt. Conversely, a moderately sized model can perform surprisingly well when context is curated carefully.

For this reason, it is often better to present users with reasoning tiers that already encode safe defaults, instead of asking them to manage all prompt and token trade-offs manually. The combination of LiteLLM routing and LibreChat model presets is particularly effective for this purpose.

5.9 Best practices

The following best practices are recommended when designing reasoning tiers in LibreChat through LiteLLM:

- expose only a few visible tiers, each with a clear semantic role;
- separate "no reasoning," "balanced reasoning," and "maximum reasoning" into distinct routes;
- align fallbacks with the quality expectations of each route;
- reserve large reasoning-enabled routes for tasks that actually benefit from them;
- keep timeout, budget, and context settings consistent with the intended usage pattern;
- treat reasoning as part of the overall system design, not as a cosmetic feature.

In summary, a good reasoning architecture in LibreChat is not built by simply turning reasoning on everywhere. It is built by assigning different reasoning profiles to different routes, exposing them with clear user-facing names, and tuning the proxy so that each route behaves consistently for the type of task it is meant to solve.

6 Web Search in LibreChat

LibreChat can extend a standard chat workflow with a live web-retrieval pipeline. At a high level, when the user asks for recent or external information, the system does not rely only on the model's internal knowledge. Instead, it performs a search on the web, gathers candidate sources, extracts useful content from the selected pages, ranks that content by relevance, and finally injects the most useful evidence into the model prompt before generating the answer.

In practical terms, this workflow can be seen as a lightweight retrieval-augmented generation process for the open web. The large language model remains the final reasoning component, but its answer is grounded in fresh documents retrieved at query time. This is especially useful for news, current affairs, market information, regulations, software updates, and any topic for which static model knowledge may be outdated.

6.1 Role of SearXNG, Serper, and Jina

The web-search stack can be split into three logical stages: *search*, *scraping*, and *reranking*.

SearXNG as search provider SearXNG is the meta-search engine used to retrieve initial candidate results. Its purpose is to answer the question: *which URLs are potentially relevant to the user's query?* It does not usually provide the final textual evidence used by the model; instead, it returns a list of candidate pages, often including titles, snippets, and links. Because SearXNG is self-hosted, it is attractive in privacy-oriented architectures and gives good control over the search layer without hard-coupling the system to a single commercial search engine.

Serper as scraper After the initial URLs are found, the system still needs readable page content. This is where the scraper comes in. In this setup, Serper is used as the scraping component: it fetches page content and turns raw web pages into text that can be consumed by downstream logic. Conceptually, the scraper answers the question: *what does each selected page actually say?* This stage is crucial because search snippets alone are too short and too noisy for reliable answer generation.

Jina as reranker Once multiple candidate passages or documents have been extracted, not all of them are equally useful. Jina is used as the reranker to score and reorder the retrieved material according to semantic relevance to the original user query. The reranker answers the question: *which pieces of retrieved evidence are the most relevant and should be kept in the final context?* This stage is extremely important because a good reranker improves both quality and efficiency: it raises answer accuracy while reducing the amount of irrelevant text sent to the model.

6.2 Why this decomposition matters

Separating the pipeline into search, scraping, and reranking is important because each component solves a different problem:

- the **search provider** maximizes recall by discovering potentially useful sources;
- the **scraper** converts URLs into usable textual evidence;
- the **reranker** maximizes precision by selecting only the most relevant evidence.

This modularity is valuable because the quality of web-grounded answers depends not only on the language model, but also on the quality of source discovery, content extraction, and evidence selection.

6.3 What is important to configure correctly

A stable LibreChat web-search setup depends less on a single parameter and more on the consistency of the whole pipeline. The most important settings can be grouped into six areas.

1. Correct provider wiring The first requirement is that LibreChat and the retrieval backend agree on which services are used. The search provider, scraper, and reranker must be explicitly configured and their credentials must be available through environment variables. A mismatch here usually leads to silent failures, partial results, or empty retrieval outputs.

2. Conservative retrieval limits A web-search system can easily generate too much text. For this reason, limits such as the number of search results, the number of final retained results, and the scraper timeout are critical. Lower values reduce recall but improve stability; higher values increase coverage but can create context bloat. In practice, these limits should be tuned conservatively first, then increased only if the model still has enough headroom.

3. Relevance filtering The reranking stage is only effective if the system is strict enough about relevance. Thresholds such as a minimum relevance score and limits on citations per source help prevent low-value or repetitive content from entering the final prompt. This is one of the most important controls for keeping the model context focused.

4. Model context budgeting Even if the retrieval stack works perfectly, the final answer can still fail if too much retrieved material is injected into the model context. For that reason, context-related settings matter at two levels:

1. the model or proxy must expose a sufficiently large input context window;
2. the application should still cap the amount of injected evidence, rather than assuming that a large-context model can safely absorb unlimited search output.

Large context windows improve robustness, but they do not remove the need for aggressive filtering and ranking.

5. Agent-specific tuning When agents are enabled, they often need stricter context controls than ordinary chats. An agent that uses web search can accumulate system instructions, tool traces, citations, and retrieved passages very quickly. In practice, per-agent context limits are often as important as global web-search settings, especially for preventing context-pruning failures.

6. Observability and debugging A production-ready setup should expose enough debugging information to understand where failures happen: search discovery, scraping, reranking, prompt assembly, or model inference. Without observability, many failures look like "the model is broken" when the real problem is excessive retrieved payload or a misconfigured provider.

6.4 Recommended operational principles

A good operational strategy is to begin with a narrow and stable configuration, then relax constraints gradually only if needed. In particular:

- keep the number of retrieved web results small at first;
- use reranking aggressively to preserve only the best evidence;
- avoid sending raw or redundant page text to the model;
- reserve larger and more capable models for web-heavy tasks;
- use lower-context agents only for focused tasks that do not require broad retrieval.

In short, the web-search feature in LibreChat works best when treated as a pipeline to be *compressed and filtered*, not merely as a way to collect as many pages as possible. Search quality is important, but context discipline is what ultimately determines whether the final answer is both accurate and stable.

7 Agents in LibreChat

LibreChat Agents are reusable, configurable assistants built inside the platform. An agent combines a persistent instruction set, a selected model, optional tools, optional files, and capability switches into a single reusable interface. In practice, an agent is useful when a recurring task should always be executed with the same role, constraints, knowledge sources, and operational behavior.

Agents are especially valuable when a chatbot evolves from a generic assistant into a collection of specialized assistants. Instead of forcing one global prompt to handle every scenario, LibreChat makes it possible to define focused agents for distinct domains such as legal drafting, software support, literature review, recipe design, internal documentation, or web-grounded news analysis.

7.1 How Agents should be used in practice

From an architectural point of view, Agents should be treated as *specialized workflows*, not merely as renamed chats. The most effective usage pattern is to assign each agent a narrow responsibility, a stable tone, and a controlled set of capabilities. This reduces prompt ambiguity, improves repeatability, and makes the system easier to maintain.

A good agent is usually designed around one of the following patterns:

- a **domain specialist**, focused on a specific body of knowledge or type of task;
- a **workflow assistant**, responsible for a repeated operational process;
- a **research agent**, combining reasoning with web search and/or file search;
- a **structured-output agent**, optimized for templates, reports, checklists, or transformations.

In contrast, broad and overloaded agents tend to become harder to debug, less predictable, and more expensive in terms of tokens.

7.2 Best practices for designing effective Agents

The most important design principle is *scope discipline*. Each agent should have a clear mission, explicit boundaries, and a concise but precise instruction set. In practice, it is better to create two focused agents than one oversized agent that tries to do everything.

Instructions should describe:

- the agent's role and domain;
- the expected output style;
- what the agent should prioritize;
- what the agent should avoid;
- how it should behave when information is missing or uncertain.

Capabilities should be enabled only when they are genuinely useful. In this document, file search can already be treated as part of the RAG workflow, while web search has been explained in the previous section. Agents can take advantage of both, but they should not automatically be enabled in every case. A narrow procedural agent may need neither. A document-grounded assistant may need file search but not web search. A current-affairs or monitoring agent may need web search but not file search. This selective approach improves reliability and reduces unnecessary prompt and tool overhead.

Files should also be attached intentionally. Large documents that must be queried semantically are better handled through file search, while small persistent reference material can often be attached as file context. Structured data may be better handled by code-oriented workflows rather than being treated as ordinary prose.

Finally, agents should always be tested with representative prompts, edge cases, and failure cases. An agent that works on one ideal prompt but fails on short, vague, or adversarial prompts is not yet production-ready.

7.3 Model tuning for Agents

One of the most useful features of LibreChat Agents is the ability to tune model behavior directly. This matters because the same instructions can behave very differently depending on the model parameters.

The most important parameters are the following.

Temperature Temperature controls how deterministic or creative the model is. Lower values generally produce more stable, repeatable, and conservative outputs, while higher values encourage variety and exploration. For procedural agents, technical assistants, support agents, or policy-sensitive agents, lower temperature is usually preferable. For ideation, brainstorming, or writing assistants, a slightly higher value can be useful.

Max context tokens Max context tokens define how much context the model is allowed to receive. This is one of the most important parameters in real-world LibreChat deployments. If it is too low, relevant instructions, citations, retrieved passages, or prior messages may be dropped. If it is too high, the system may become slower, more expensive, and more vulnerable to context bloat. For agents that use web search or file-heavy workflows, context budgeting is critical.

Max output tokens Max output tokens cap the size of the final answer. If this value is too small, responses may be cut off or overly compressed. If it is too large, answers may become verbose, expensive, or less controlled. In practice, procedural agents usually benefit from moderate output limits, while report-writing or research agents may require higher values.

Top-p Top-p controls nucleus sampling, that is, how much of the probability distribution is considered when selecting the next token. In many deployments it is left at its default value, especially when temperature is already sufficient. It becomes useful when one wants to fine-tune stylistic variability without relying only on temperature.

Presence and frequency penalties These penalties influence repetition behavior. Presence penalty encourages the model to explore new topics instead of returning to the same wording or ideas. Frequency penalty reduces repeated tokens and can make outputs less redundant. They are optional, but can be helpful for agents that tend to repeat themselves in long-form answers.

Reasoning effort For models that support it, reasoning effort controls how much deliberate reasoning the model performs. Lower settings are faster and lighter; higher settings can improve difficult multi-step tasks, but may increase latency and token usage. It is usually worth increasing this setting only for complex analysis, planning, or synthesis tasks.

Verbosity Verbosity controls how long and detailed the answer should be. This can be useful when an agent should consistently produce compact operational outputs rather than expansive prose.

7.4 When parameter tuning is especially important

Parameter tuning matters most when an agent uses tools that inject additional material into the prompt. Web search is the clearest example. A web-enabled agent may accumulate system instructions, user instructions, search results, scraped passages, reranked evidence, citations, and conversation history within a single request. In such a situation, `maxContextTokens` becomes a practical stability parameter, not just a theoretical one.

A common failure pattern in tool-augmented systems is to assume that a large-context model alone solves the problem. In reality, even strong models can fail if too much low-value material is injected into the final prompt. For this reason, web-search agents usually benefit from:

- stricter retrieval limits;

- stronger relevance filtering;
- moderate output budgets;
- a context window large enough for retrieved evidence, but still bounded conservatively.

This is particularly important when the agent must combine multiple capabilities. In other words, stable agent design is not only about choosing a large model, but also about controlling how much context the agent is allowed to consume.

7.5 Recommended deployment strategy

A practical deployment strategy is to map agent design to task complexity:

- use **small or medium models** for narrow agents with stable instructions and little or no external retrieval;
- use **larger models** for agents that combine reasoning with web search, broad synthesis, or multi-document analysis;
- keep **nonessential capabilities disabled** by default;
- prefer **one capability-rich agent only when the workflow truly requires it**;
- separate **document-grounded agents** from **web-grounded agents** whenever that separation makes behavior easier to control.

This modular strategy usually produces better reliability than attempting to create a single universal agent.

7.6 Operational and security considerations

Agents should also be reviewed from a governance perspective. Before sharing an agent, its instructions, attached files, and enabled capabilities should be checked carefully. Any persistent context or attached material should be treated as potentially exposable through conversation. For this reason, shared agents should only include information that is appropriate for their intended audience.

7.7 Templates and examples in this code base

In this code base, the directory `templates/agents` contains example Markdown templates that can be used as starting points for building effective agents. These templates are useful because they make it easier to standardize agent structure, reduce prompt-writing inconsistency, and accelerate experimentation with specialized assistants.

For that reason, a recommended workflow is to begin with one of the templates in `templates/agents`, adapt it to a narrowly defined task, enable only the necessary capabilities, and then tune model parameters only after the baseline behavior has been validated.